C.N. PRACTICAL EXAM KEY ANSWERS

BATCH - 2 | All Programs (Text Format)

1. TCP Client-Server File Transfer (FileServer.java & FileClient.java)

SERVER — FileServer.java

```
import java.io.*;
import java.net.*;

public class FileServer {
public static void main(String[] args) {
try {
ServerSocket serverSocket = new ServerSocket(5000);
System.out.println("Server is waiting for connection...");

Socket socket = serverSocket.accept();
System.out.println("Client connected!");

File file = new File("test.txt");
FileInputStream fis = new FileInputStream(file);
OutputStream os = socket.getOutputStream();

byte[] buffer = new byte[1024];
int bytesRead;

while ((bytesRead = fis.read(buffer)) > 0) {
os.write(buffer, 0, bytesRead);
}

System.out.println("File sent successfully!");

fis.close();
os.close();
socket.close();
serverSocket.close();

} catch (Exception e) {
System.out.println(e);
}
}
}
```

CLIENT — FileClient.java

```
import java.io.*;
```

```
import java.net.*;

public class FileClient {
public static void main(String[] args) {
try {
Socket socket = new Socket("localhost", 5000);
System.out.println("Connected to server!");

InputStream is = socket.getInputStream();
FileOutputStream fos = new FileOutputStream("received.txt");

byte[] buffer = new byte[1024];
int bytesRead;

while ((bytesRead = is.read(buffer)) > 0) {
fos.write(buffer, 0, bytesRead);
}

System.out.println("File received successfully!");

fos.close();
is.close();
socket.close();

} catch (Exception e) {
System.out.println(e);
}
}
}
```

2. Download Webpage Using TCP Socket (WebPageDownloader.java)

```
import java.io.*;
import java.net.*;

public class WebPageDownloader {
public static void main(String[] args) {
try {
String host = "example.com"; // website
String path = "/"; // page path

// create socket (TCP)
Socket socket = new Socket(host, 80);

// send HTTP request
PrintWriter pw = new PrintWriter(socket.getOutputStream(), true);
pw.println("GET " + path + " HTTP/1.1");
pw.println("Host: " + host);
pw.println("Connection: close");
```

```
pw.println(); // blank line

// read response
BufferedReader br = new BufferedReader(
new InputStreamReader(socket.getInputStream()));

// write to html file
BufferedWriter bw = new BufferedWriter(
new FileWriter("output.html"));

String line;
while ((line = br.readLine()) != null) {
bw.write(line);
bw.newLine();
}

System.out.println("Webpage downloaded successfully!");

// close resources
bw.close();
br.close();
pw.close();
socket.close();

} catch (Exception e) {
e.printStackTrace();
}
}
}
```

3. TCP Client-Server Chat Application (ChatServer.java & ChatClient.java)

SERVER — ChatServer.java

```
import java.io.*;
import java.net.*;

public class ChatServer {
public static void main(String[] args) {
try {
ServerSocket ss = new ServerSocket(5000);
System.out.println("Server waiting...");

Socket s = ss.accept();
System.out.println("Client connected!");

DataInputStream dis = new DataInputStream(s.getInputStream());
DataOutputStream dos = new DataOutputStream(s.getOutputStream());
```

```
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

String msg = "", reply = "";

while (!msg.equals("exit")) {
msg = dis.readUTF();
System.out.println("Client: " + msg);

System.out.print("Server: ");
reply = br.readLine();
dos.writeUTF(reply);
}

dis.close();
dos.close();
s.close();
ss.close();

} catch (Exception e) {
e.printStackTrace();
}
}
}
```

CLIENT — ChatClient.java

```
import java.io.*;
import java.net.*;

public class ChatClient {
public static void main(String[] args) {
try {
Socket s = new Socket("localhost", 5000);
System.out.println("Connected to server!");

DataInputStream dis = new DataInputStream(s.getInputStream());
DataOutputStream dos = new DataOutputStream(s.getOutputStream());

BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

String msg = "", reply = "";

while (!msg.equals("exit")) {
System.out.print("Client: ");
msg = br.readLine();
dos.writeUTF(msg);

reply = dis.readUTF();
System.out.println("Server: " + reply);
}

```

```
dis.close();
dos.close();
s.close();

} catch (Exception e) {
e.printStackTrace();
}
}
}
```

4. Hello World TCP Client-Server (HelloServer.java & HelloClient.java)

SERVER — HelloServer.java

```
import java.io.*;
import java.net.*;

public class HelloServer {
public static void main(String[] args) {
try {
ServerSocket ss = new ServerSocket(5000);
System.out.println("Server waiting...");

Socket s = ss.accept();
System.out.println("Client connected");

DataInputStream dis = new DataInputStream(s.getInputStream());

String message = dis.readUTF();
System.out.println("Message from client: " + message);

dis.close();
s.close();
ss.close();

} catch (Exception e) {
e.printStackTrace();
}
}
}
```

CLIENT — HelloClient.java

```
import java.io.*;
import java.net.*;

public class HelloClient {
public static void main(String[] args) {
try {
Socket s = new Socket("localhost", 5000);
```

```
DataOutputStream dos = new DataOutputStream(s.getOutputStream());

dos.writeUTF("Hello, world!");

dos.close();
s.close();

} catch (Exception e) {
e.printStackTrace();
}
}
}
```

5. DNS Simulation Using UDP Socket (DNSServer.java & DNSClient.java)

SERVER — DNSServer.java

```
import java.net.*;

public class DNSServer {
public static void main(String[] args) {
try {
DatagramSocket ds = new DatagramSocket(5000);
System.out.println("DNS Server is running...");

byte[] receive = new byte[1024];

DatagramPacket dp = new DatagramPacket(receive, receive.length);
ds.receive(dp);

String request = new String(dp.getData()).trim();
System.out.println("Received domain: " + request);

// Simulated DNS mapping
String response;
if (request.equals("google.com"))
response = "142.250.190.78";
else if (request.equals("yahoo.com"))
response = "98.137.11.163";
else
response = "Domain not found";

byte[] send = response.getBytes();

DatagramPacket dpSend = new DatagramPacket(
send, send.length,
dp.getAddress(), dp.getPort());
```

```
ds.send(dpSend);

System.out.println("Response sent");

ds.close();

} catch (Exception e) {
e.printStackTrace();
}
}
}
```

CLIENT — DNSClient.java

```
import java.net.*;
import java.util.Scanner;

public class DNSClient {
public static void main(String[] args) {
try {
Scanner sc = new Scanner(System.in);

System.out.print("Enter frame size: ");
int frameSize = sc.nextInt();
sc.nextLine();

System.out.print("Enter domain name: ");
String domain = sc.nextLine();

// pad to frame size
StringBuilder frame = new StringBuilder(domain);
while (frame.length() < frameSize) {
frame.append(" "); // padding
}

byte[] send = frame.toString().getBytes();

DatagramSocket ds = new DatagramSocket();

InetAddress ip = InetAddress.getByName("localhost");

DatagramPacket dp = new DatagramPacket(send, send.length, ip, 5000);
ds.send(dp);

byte[] receive = new byte[1024];
DatagramPacket dpReceive = new DatagramPacket(receive, receive.length);

ds.receive(dpReceive);

String response = new String(dpReceive.getData()).trim();

```

```
System.out.println("IP Address: " + response);

ds.close();
sc.close();

} catch (Exception e) {
e.printStackTrace();
}
}
}
```

6. File Transfer with User-Specified File (FileServer.java & FileClient.java)

SERVER — FileServer.java

```
import java.io.*;
import java.net.*;

public class FileServer {
public static void main(String[] args) {
try {
ServerSocket ss = new ServerSocket(5000);
System.out.println("Server waiting...");

Socket s = ss.accept();
System.out.println("Client connected");

DataInputStream dis = new DataInputStream(s.getInputStream());
DataOutputStream dos = new DataOutputStream(s.getOutputStream());

// receive filename
String fileName = dis.readUTF();
File file = new File(fileName);

if (file.exists()) {
dos.writeUTF("FOUND");
dos.writeLong(file.length());

FileInputStream fis = new FileInputStream(file);
byte[] buffer = new byte[4096];
int count;

while ((count = fis.read(buffer)) != -1) {
dos.write(buffer, 0, count);
}

fis.close();
System.out.println("File sent: " + fileName);
```

```
} else {
dos.writeUTF("NOTFOUND");
System.out.println("File not found: " + fileName);
}

dos.flush();

dis.close();
dos.close();
s.close();
ss.close();

} catch (Exception e) {
e.printStackTrace();
}
}
}
```

CLIENT — FileClient.java

```
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class FileClient {
public static void main(String[] args) {
try {
Socket s = new Socket("localhost", 5000);
System.out.println("Connected to server");

DataInputStream dis = new DataInputStream(s.getInputStream());
DataOutputStream dos = new DataOutputStream(s.getOutputStream());
Scanner sc = new Scanner(System.in);

System.out.print("Enter file name: ");
String fileName = sc.nextLine();

// send filename
dos.writeUTF(fileName);

String status = dis.readUTF();

if (status.equals("FOUND")) {
long fileSize = dis.readLong();

FileOutputStream fos = new FileOutputStream("downloaded_" + fileName);

byte[] buffer = new byte[4096];
int count;
long remaining = fileSize;
```

```
while ((count = dis.read(buffer, 0, (int)Math.min(buffer.length, remaining))) > 0) {
fos.write(buffer, 0, count);
remaining -= count;
if (remaining == 0) break;
}

fos.close();
System.out.println("File downloaded successfully");

} else {
System.out.println("File not found on server");
}

dis.close();
dos.close();
s.close();
sc.close();

} catch (Exception e) {
e.printStackTrace();
}
}
}
```

7. Capture FTP Username & Password Using Wireshark

PROCEDURE (No Java program — done using Wireshark tool):

```
Step 1: Open Wireshark
- Open Wireshark application
- Select network interface (Wi-Fi / Ethernet)
- Click Start Capture

Step 2: Connect to FTP server in another terminal:
ftp localhost

Step 3: Enter credentials:
Username: test
Password: test123

Step 4: Go back to Wireshark -> Stop capture

Step 5: In filter bar, type:
ftp.request.command == "USER" || ftp.request.command == "PASS"

Step 6: Click packets -> Expand:
File Transfer Protocol

OUTPUT in Wireshark packet details:
```

```
Request: USER test
Request: PASS test123
Username and password are visible in plaintext
```

8. RARP Simulation Using UDP (RARPServer.java & RARPClient.java)

SERVER — RARPServer.java

```
import java.net.*;

public class RARPServer {
public static void main(String[] args) {
try {
DatagramSocket ds = new DatagramSocket(5000);
System.out.println("RARP Server is running...");

byte[] receive = new byte[1024];
DatagramPacket dp = new DatagramPacket(receive, receive.length);

ds.receive(dp);

String mac = new String(dp.getData()).trim();
System.out.println("Received MAC: " + mac);

String ip;

// Simulated RARP table
if (mac.equals("AA:BB:CC:DD:EE:01"))
ip = "192.168.1.1";
else if (mac.equals("AA:BB:CC:DD:EE:02"))
ip = "192.168.1.2";
else
ip = "IP not found";

byte[] send = ip.getBytes();

DatagramPacket dpSend = new DatagramPacket(
send, send.length,
dp.getAddress(), dp.getPort());

ds.send(dpSend);

System.out.println("Response sent");

ds.close();

} catch (Exception e) {
e.printStackTrace();
}
```

```
}
}
```

CLIENT — RARPClient.java

```
import java.net.*;
import java.util.Scanner;

public class RARPClient {
public static void main(String[] args) {
try {
Scanner sc = new Scanner(System.in);

DatagramSocket ds = new DatagramSocket();

InetAddress ip = InetAddress.getByName("localhost");

System.out.print("Enter MAC Address: ");
String mac = sc.nextLine();

byte[] send = mac.getBytes();

DatagramPacket dp = new DatagramPacket(send, send.length, ip, 5000);
ds.send(dp);

byte[] receive = new byte[1024];
DatagramPacket dpReceive = new DatagramPacket(receive, receive.length);

ds.receive(dpReceive);

String response = new String(dpReceive.getData()).trim();

System.out.println("IP Address: " + response);

ds.close();
sc.close();

} catch (Exception e) {
e.printStackTrace();
}
}
}
```

9. Distance Vector Routing Algorithm (DistanceVectorRouting.java)

```
import java.util.*;

public class DistanceVectorRouting {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
```

```
System.out.print("Enter number of nodes: ");
int n = sc.nextInt();

int[][] cost = new int[n][n];

System.out.println("Enter cost matrix (use 999 for infinity):");
for (int i = 0; i < n; i++) {
for (int j = 0; j < n; j++) {
cost[i][j] = sc.nextInt();
}
}

int[][] dist = new int[n][n];

// initialize distance table
for (int i = 0; i < n; i++) {
for (int j = 0; j < n; j++) {
dist[i][j] = cost[i][j];
}
}

// Distance Vector Algorithm
for (int k = 0; k < n; k++) {
for (int i = 0; i < n; i++) {
for (int j = 0; j < n; j++) {
if (dist[i][k] + dist[k][j] < dist[i][j]) {
dist[i][j] = dist[i][k] + dist[k][j];
}
}
}
}

System.out.println("\nShortest distance matrix:");
for (int i = 0; i < n; i++) {
for (int j = 0; j < n; j++) {
System.out.print(dist[i][j] + "\t");
}
System.out.println();
}

sc.close();
}
}
```

10. Congestion Control - Leaky Bucket Algorithm (LeakyBucket.java)

```
import java.util.*;

```

```
public class LeakyBucket {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);

System.out.print("Enter bucket size: ");
int bucketSize = sc.nextInt();

System.out.print("Enter output rate: ");
int outputRate = sc.nextInt();

System.out.print("Enter number of packets: ");
int n = sc.nextInt();

int[] packets = new int[n];

System.out.println("Enter packet sizes:");
for (int i = 0; i < n; i++) {
packets[i] = sc.nextInt();
}

int bucket = 0;

System.out.println("\nTime\tIncoming\tBucket\tOutput\tDropped");

for (int i = 0; i < n; i++) {
int incoming = packets[i];
int dropped = 0;

if (bucket + incoming > bucketSize) {
dropped = (bucket + incoming) - bucketSize;
incoming -= dropped;
}

bucket += incoming;

int output = Math.min(bucket, outputRate);
bucket -= output;

System.out.println((i+1) + "\t" + packets[i] + "\t\t" +
bucket + "\t" + output + "\t" + dropped);
}

sc.close();
}
}
```

11. Sliding Window Throughput Analysis (SlidingWindowThroughput.java)

```
import java.util.*;
```

```
public class SlidingWindowThroughput {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);

System.out.print("Enter window size: ");
int windowSize = sc.nextInt();

System.out.print("Enter number of test cases: ");
int t = sc.nextInt();

System.out.println("\nPktSize\tRTT\tErrorRate\tThroughput");

for (int i = 0; i < t; i++) {
System.out.println("\nTest Case " + (i + 1));

System.out.print("Enter packet size (bits): ");
double packetSize = sc.nextDouble();

System.out.print("Enter RTT (seconds): ");
double rtt = sc.nextDouble();

System.out.print("Enter error rate (0 to 1): ");
double errorRate = sc.nextDouble();

double success = (1 - errorRate);

double throughput = (windowSize * packetSize * success) / rtt;

System.out.println(packetSize + "\t" + rtt + "\t" +
errorRate + "\t\t" + throughput + " bits/sec");
}

sc.close();
}
}
```

12. Distance Vector Routing vs Link State Routing Comparison

Distance Vector Routing — DVR.java

```
import java.util.*;

public class DVR {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);

System.out.print("Enter number of nodes: ");
int n = sc.nextInt();
```

```
int[][] dist = new int[n][n];

System.out.println("Enter cost matrix (999 for infinity):");
for (int i = 0; i < n; i++) {
for (int j = 0; j < n; j++) {
dist[i][j] = sc.nextInt();
}
}

for (int k = 0; k < n; k++) {
for (int i = 0; i < n; i++) {
for (int j = 0; j < n; j++) {
if (dist[i][k] + dist[k][j] < dist[i][j]) {
dist[i][j] = dist[i][k] + dist[k][j];
}
}
}
}

System.out.println("\nDVR Shortest Path:");
for (int i = 0; i < n; i++) {
for (int j = 0; j < n; j++) {
System.out.print(dist[i][j] + "\t");
}
System.out.println();
}

sc.close();
}
}
```

Link State Routing (Dijkstra) — LinkStateRouting.java

```
import java.util.*;

public class LinkStateRouting {
static int minDistance(int[] dist, boolean[] visited, int n) {
int min = Integer.MAX_VALUE, index = -1;

for (int i = 0; i < n; i++) {
if (!visited[i] && dist[i] <= min) {
min = dist[i];
index = i;
}
}
return index;
}

public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
```

```
System.out.print("Enter number of nodes: ");
int n = sc.nextInt();

int[][] graph = new int[n][n];

System.out.println("Enter cost matrix (0 if no edge):");
for (int i = 0; i < n; i++) {
for (int j = 0; j < n; j++) {
graph[i][j] = sc.nextInt();
}
}

System.out.print("Enter source node: ");
int src = sc.nextInt();

int[] dist = new int[n];
boolean[] visited = new boolean[n];

Arrays.fill(dist, Integer.MAX_VALUE);
dist[src] = 0;

for (int count = 0; count < n - 1; count++) {
int u = minDistance(dist, visited, n);
visited[u] = true;

for (int v = 0; v < n; v++) {
if (!visited[v] && graph[u][v] != 0 &&
dist[u] + graph[u][v] < dist[v]) {
dist[v] = dist[u] + graph[u][v];
}
}
}

System.out.println("\nLink State (Dijkstra) Shortest Paths:");
for (int i = 0; i < n; i++) {
System.out.println("To node " + i + " = " + dist[i]);
}

sc.close();
}
}
```

13. Error Correction Using Hamming Code (HammingCode.java)

```
import java.util.*;

public class HammingCode {
```

```
static int[] data = new int[7];

static void calculateParityBits() {
data[0] = data[2] ^ data[4] ^ data[6]; // p1
data[1] = data[2] ^ data[5] ^ data[6]; // p2
data[3] = data[4] ^ data[5] ^ data[6]; // p4
}

static int checkError() {
int p1 = data[0] ^ data[2] ^ data[4] ^ data[6];
int p2 = data[1] ^ data[2] ^ data[5] ^ data[6];
int p4 = data[3] ^ data[4] ^ data[5] ^ data[6];

return (p4 * 4 + p2 * 2 + p1);
}

public static void main(String[] args) {
Scanner sc = new Scanner(System.in);

System.out.println("Enter 4-bit data:");
int d1 = sc.nextInt();
int d2 = sc.nextInt();
int d3 = sc.nextInt();
int d4 = sc.nextInt();

// placing data bits
data[2] = d1;
data[4] = d2;
data[5] = d3;
data[6] = d4;

calculateParityBits();

System.out.println("\nTransmitted Code:");
for (int i = 0; i < 7; i++) {
System.out.print(data[i] + " ");
}

// simulate error
System.out.println("\n\n--- Introducing Error at position 3 ---");
data[2] = data[2] ^ 1; // flip bit

System.out.println("Received Code:");
for (int i = 0; i < 7; i++) {
System.out.print(data[i] + " ");
}

int errorPos = checkError();
```

```
if (errorPos == 0) {
System.out.println("\nNo Error Detected");
} else {
System.out.println("\nError at position: " + errorPos);

// correct error
data[errorPos - 1] ^= 1;

System.out.println("Corrected Code:");
for (int i = 0; i < 7; i++) {
System.out.print(data[i] + " ");
}
}

sc.close();
}
}
```

14. CRC Error Detection (CRC.java)

```
import java.util.*;

public class CRC {

// XOR operation
static String xor(String a, String b) {
StringBuilder result = new StringBuilder();

for (int i = 1; i < b.length(); i++) {
result.append(a.charAt(i) == b.charAt(i) ? '0' : '1');
}
return result.toString();
}

// Division method
static String divide(String dividend, String divisor) {
int pick = divisor.length();

String temp = dividend.substring(0, pick);

while (pick < dividend.length()) {
if (temp.charAt(0) == '1')
temp = xor(divisor, temp) + dividend.charAt(pick);
else
temp = xor("0".repeat(pick), temp) + dividend.charAt(pick);

pick++;
}
```

```
if (temp.charAt(0) == '1')
temp = xor(divisor, temp);
else
temp = xor("0".repeat(pick), temp);

return temp;
}

public static void main(String[] args) {
Scanner sc = new Scanner(System.in);

System.out.print("Enter data bits: ");
String data = sc.nextLine();

System.out.print("Enter generator: ");
String gen = sc.nextLine();

int genLen = gen.length();

// Append zeros
String appendedData = data + "0".repeat(genLen - 1);

String remainder = divide(appendedData, gen);

String codeword = data + remainder;

System.out.println("\nTransmitted Codeword: " + codeword);

// --------- NO ERROR CASE ---------
String check = divide(codeword, gen);

if (check.contains("1"))
System.out.println("Error detected in transmission (NO ERROR CASE CHECK FAILED)");
else
System.out.println("No error detected (Correct transmission)");

// --------- ERROR SIMULATION ---------
char[] errorData = codeword.toCharArray();
errorData[2] = (errorData[2] == '0') ? '1' : '0'; // flip a bit

String corrupted = new String(errorData);

System.out.println("\nCorrupted Codeword: " + corrupted);

String check2 = divide(corrupted, gen);

if (check2.contains("1"))
System.out.println("Error detected in corrupted transmission");
else
```

```
System.out.println("No error detected");

sc.close();
}
}
```

15. ARP Protocol Simulation (ARPServer.java & ARPClient.java)

SERVER — ARPServer.java

```
import java.io.*;
import java.net.*;
import java.util.*;

public class ARPServer {
public static void main(String[] args) {
try {
ServerSocket ss = new ServerSocket(5000);
System.out.println("ARP Server is running...");

// ARP Table (IP -> MAC)
Map<String, String> arpTable = new HashMap<>();
arpTable.put("192.168.1.1", "AA:BB:CC:DD:EE:01");
arpTable.put("192.168.1.2", "AA:BB:CC:DD:EE:02");
arpTable.put("192.168.1.3", "AA:BB:CC:DD:EE:03");

Socket s = ss.accept();
System.out.println("Client connected");

DataInputStream dis = new DataInputStream(s.getInputStream());
DataOutputStream dos = new DataOutputStream(s.getOutputStream());

String ip = dis.readUTF();
System.out.println("Requested IP: " + ip);

String mac = arpTable.getOrDefault(ip, "MAC NOT FOUND");

dos.writeUTF(mac);

dis.close();
dos.close();
s.close();
ss.close();

} catch (Exception e) {
e.printStackTrace();
}
}
}
```

CLIENT — ARPClient.java

```
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class ARPClient {
public static void main(String[] args) {
try {
Socket s = new Socket("localhost", 5000);
System.out.println("Connected to ARP Server");

DataInputStream dis = new DataInputStream(s.getInputStream());
DataOutputStream dos = new DataOutputStream(s.getOutputStream());

Scanner sc = new Scanner(System.in);

System.out.print("Enter IP Address: ");
String ip = sc.nextLine();

dos.writeUTF(ip);

String mac = dis.readUTF();

System.out.println("MAC Address: " + mac);

dis.close();
dos.close();
s.close();
sc.close();

} catch (Exception e) {
e.printStackTrace();
}
}
}
```

16. UDP Datagram Socket – Server to Client Message (UDPServer.java & UDPClient.java)

SERVER — UDPServer.java

```
import java.net.*;
import java.util.Scanner;

public class UDPServer {
public static void main(String[] args) {
try {
DatagramSocket ds = new DatagramSocket();
```

```
Scanner sc = new Scanner(System.in);

System.out.print("Enter message to send: ");
String msg = sc.nextLine();

byte[] sendData = msg.getBytes();

InetAddress ip = InetAddress.getByName("localhost");

DatagramPacket dp = new DatagramPacket(
sendData,
sendData.length,
ip,
5000
);

ds.send(dp);

System.out.println("Message sent to client");

ds.close();
sc.close();

} catch (Exception e) {
e.printStackTrace();
}
}
}
```

CLIENT — UDPClient.java

```
import java.net.*;

public class UDPClient {
public static void main(String[] args) {
try {
DatagramSocket ds = new DatagramSocket(5000);

byte[] receive = new byte[1024];

DatagramPacket dp = new DatagramPacket(receive, receive.length);

System.out.println("Client waiting for message...");

ds.receive(dp);

String msg = new String(dp.getData()).trim();

System.out.println("Message from server: " + msg);
```

```
ds.close();

} catch (Exception e) {
e.printStackTrace();
}
}
}
```

17. SMTP Client Program (SMTPServer.java & SMTPClient.java)

SERVER — SMTPServer.java

```
import java.io.*;
import java.net.*;

public class SMTPServer {
public static void main(String[] args) {
try {
ServerSocket ss = new ServerSocket(5000);
System.out.println("SMTP Server started...");

Socket s = ss.accept();
System.out.println("Client connected");

BufferedReader br = new BufferedReader(
new InputStreamReader(s.getInputStream()));

PrintWriter pw = new PrintWriter(s.getOutputStream(), true);

pw.println("220 SMTP Server Ready");

String msg;

while ((msg = br.readLine()) != null) {
System.out.println("Client: " + msg);

if (msg.startsWith("HELO"))
pw.println("250 OK Hello");
else if (msg.startsWith("MAIL FROM"))
pw.println("250 OK Sender");
else if (msg.startsWith("RCPT TO"))
pw.println("250 OK Receiver");
else if (msg.equals("DATA"))
pw.println("354 Start Input");
else if (msg.equals("."))
pw.println("250 Message Received");
else if (msg.equals("QUIT")) {
pw.println("221 Bye");
break;
```

```
} else {
pw.println("250 OK");
}
}

s.close();
ss.close();

} catch (Exception e) {
e.printStackTrace();
}
}
}
```

CLIENT — SMTPClient.java

```
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class SMTPClient {
public static void main(String[] args) {
try {
Socket socket = new Socket("localhost", 5000);

BufferedReader in = new BufferedReader(
new InputStreamReader(socket.getInputStream()));

PrintWriter out = new PrintWriter(socket.getOutputStream(), true);

Scanner sc = new Scanner(System.in);

System.out.println(in.readLine());

out.println("HELO localhost");
System.out.println(in.readLine());

System.out.print("Sender: ");
String from = sc.nextLine();
out.println("MAIL FROM:<" + from + ">");
System.out.println(in.readLine());

System.out.print("Receiver: ");
String to = sc.nextLine();
out.println("RCPT TO:<" + to + ">");
System.out.println(in.readLine());

out.println("DATA");
System.out.println(in.readLine());

System.out.println("Type message (end with .):");
```

```
String msg;
while (!(msg = sc.nextLine()).equals(".")) {
out.println(msg);
}

out.println(".");
System.out.println(in.readLine());

out.println("QUIT");
System.out.println(in.readLine());

socket.close();
sc.close();

} catch (Exception e) {
e.printStackTrace();
}
}
}
```

18. RARP Protocol Simulation using TCP (RARPServer.java & RARPClient.java)

SERVER — RARPServer.java

```
import java.io.*;
import java.net.*;
import java.util.*;

public class RARPServer {
public static void main(String[] args) {
try {
ServerSocket ss = new ServerSocket(5000);
System.out.println("RARP Server started...");

// MAC -> IP table
Map<String, String> rarpTable = new HashMap<>();
rarpTable.put("AA:BB:CC:DD:EE:01", "192.168.1.1");
rarpTable.put("AA:BB:CC:DD:EE:02", "192.168.1.2");
rarpTable.put("AA:BB:CC:DD:EE:03", "192.168.1.3");

Socket s = ss.accept();
System.out.println("Client connected");

BufferedReader br = new BufferedReader(
new InputStreamReader(s.getInputStream()));

PrintWriter pw = new PrintWriter(s.getOutputStream(), true);
```

```
String mac = br.readLine();
System.out.println("Received MAC: " + mac);

String ip = rarpTable.getOrDefault(mac, "IP NOT FOUND");

pw.println(ip);

br.close();
pw.close();
s.close();
ss.close();

} catch (Exception e) {
e.printStackTrace();
}
}
}
```

CLIENT — RARPClient.java

```
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class RARPClient {
public static void main(String[] args) {
try {
Socket s = new Socket("localhost", 5000);
System.out.println("Connected to RARP Server");

BufferedReader br = new BufferedReader(
new InputStreamReader(s.getInputStream()));

PrintWriter pw = new PrintWriter(s.getOutputStream(), true);

Scanner sc = new Scanner(System.in);

System.out.print("Enter MAC Address: ");
String mac = sc.nextLine();

pw.println(mac);

String ip = br.readLine();

System.out.println("IP Address: " + ip);

br.close();
pw.close();
s.close();
sc.close();
```

```
} catch (Exception e) {
e.printStackTrace();
}
}
}
```

19. HTTP Web Client – Download Webpage Using TCP Socket (HTTPClient.java)

```
import java.io.*;
import java.net.*;

public class HTTPClient {
public static void main(String[] args) {
try {
String host = "example.com"; // website
String path = "/"; // webpage path

Socket socket = new Socket(host, 80);

PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
BufferedReader in = new BufferedReader(
new InputStreamReader(socket.getInputStream()));

// HTTP GET request
out.println("GET " + path + " HTTP/1.1");
out.println("Host: " + host);
out.println("Connection: close");
out.println();

// write response to file
BufferedWriter file = new BufferedWriter(
new FileWriter("webpage.html"));

String line;
while ((line = in.readLine()) != null) {
file.write(line);
file.newLine();
}

System.out.println("Webpage downloaded successfully!");

file.close();
in.close();
out.close();
socket.close();

} catch (Exception e) {
e.printStackTrace();
}
```

```
}
}
```

20. Network Layer Packet Capture Simulation (NetworkLayerSimulation.java)

```
import java.util.*;

class Packet {
String src, dest, data;

Packet(String src, String dest, String data) {
this.src = src;
this.dest = dest;
this.data = data;
}
}

class Network {
Random rand = new Random();

void sendPacket(Packet p) {
System.out.println("s PACKET sent from " + p.src + " to " + p.dest);

// simulate network condition
int chance = rand.nextInt(100);

if (chance < 70) { // 70% success
System.out.println("r PACKET received at " + p.dest);
} else { // 30% drop
System.out.println("d PACKET dropped in network");
}
}
}

public class NetworkLayerSimulation {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
Network net = new Network();

System.out.print("Enter source: ");
String src = sc.nextLine();

System.out.print("Enter destination: ");
String dest = sc.nextLine();

System.out.print("Enter message: ");
String msg = sc.nextLine();

Packet p = new Packet(src, dest, msg);
```

```
System.out.println("\n--- Network Simulation Start ---\n");

for (int i = 0; i < 5; i++) {
net.sendPacket(p);
}

System.out.println("\n--- Simulation End ---");

sc.close();
}
}
```

21. TCP vs UDP Performance Analysis (TCPvsUDP.java)

```
import java.util.*;

public class TCPvsUDP {

static Random rand = new Random();

// TCP Simulation
static void tcpSimulation(int packets) {
System.out.println("\n--- TCP Simulation ---");

int delivered = 0;

for (int i = 1; i <= packets; i++) {
try {
Thread.sleep(100); // simulate delay
} catch (Exception e) {}

System.out.println("Packet " + i + " sent and ACK received (TCP)");
delivered++;
}

System.out.println("TCP Delivered: " + delivered + "/" + packets);
}

// UDP Simulation
static void udpSimulation(int packets) {
System.out.println("\n--- UDP Simulation ---");

int delivered = 0;
int lost = 0;

for (int i = 1; i <= packets; i++) {

int chance = rand.nextInt(100);
```

```
if (chance < 75) { // 75% success
System.out.println("Packet " + i + " received (UDP)");
delivered++;
} else {
System.out.println("Packet " + i + " lost (UDP)");
lost++;
}

try {
Thread.sleep(50); // faster than TCP
} catch (Exception e) {}
}

System.out.println("UDP Delivered: " + delivered + "/" + packets);
System.out.println("UDP Lost: " + lost);
}

public static void main(String[] args) {
Scanner sc = new Scanner(System.in);

System.out.print("Enter number of packets: ");
int packets = sc.nextInt();

tcpSimulation(packets);
udpSimulation(packets);

System.out.println("\n--- PERFORMANCE COMPARISON ---");
System.out.println("TCP: Reliable but slower");
System.out.println("UDP: Faster but may lose packets");

sc.close();
}
}
```

23. HDLC Frame – Bit Stuffing & Character Stuffing (HDLCStuffing.java)

```
import java.util.*;

public class HDLCStuffing {

// ---------------- BIT STUFFING ----------------
static void bitStuffing(String data) {
StringBuilder result = new StringBuilder();
int count = 0;

for (int i = 0; i < data.length(); i++) {
char bit = data.charAt(i);
result.append(bit);
```

```
if (bit == '1') {
count++;
} else {
count = 0;
}

// after 5 consecutive 1s, insert 0
if (count == 5) {
result.append('0');
count = 0;
}
}

System.out.println("\nBit Stuffed Data: " + result);
}

// ---------------- CHARACTER STUFFING ----------------
static void charStuffing(String data) {
String flag = "F";
String escape = "E";

StringBuilder result = new StringBuilder();

result.append(flag); // start flag

for (int i = 0; i < data.length(); i++) {
String ch = String.valueOf(data.charAt(i));

if (ch.equals(flag) || ch.equals(escape)) {
result.append(escape);
}

result.append(ch);
}

result.append(flag); // end flag

System.out.println("Character Stuffed Frame: " + result);
}

public static void main(String[] args) {
Scanner sc = new Scanner(System.in);

System.out.print("Enter binary data (Bit Stuffing): ");
String bitData = sc.nextLine();

System.out.print("Enter character data (Char Stuffing): ");
String charData = sc.nextLine();
```

```
bitStuffing(bitData);
charStuffing(charData);

sc.close();
}
}
```

24. Date & Time Server-Client (DateTimeServer.java & DateTimeClient.java)

SERVER — DateTimeServer.java

```
import java.io.*;
import java.net.*;
import java.time.*;

public class DateTimeServer {
public static void main(String[] args) {
try {
ServerSocket ss = new ServerSocket(5000);
System.out.println("Server started...");

Socket s = ss.accept();
System.out.println("Client connected");

PrintWriter out = new PrintWriter(s.getOutputStream(), true);

String dateTime = LocalDateTime.now().toString();

out.println("Current Date & Time: " + dateTime);

s.close();
ss.close();

} catch (Exception e) {
e.printStackTrace();
}
}
}
```

CLIENT — DateTimeClient.java

```
import java.io.*;
import java.net.*;

public class DateTimeClient {
public static void main(String[] args) {
try {
Socket s = new Socket("localhost", 5000);

```

```
BufferedReader in = new BufferedReader(
new InputStreamReader(s.getInputStream()));

String msg = in.readLine();

System.out.println("Message from server:");
System.out.println(msg);

s.close();

} catch (Exception e) {
e.printStackTrace();
}
}
}
```

25. TCP Performance Analysis (TCPPerformance.java)

```
import java.util.*;

public class TCPPerformance {

public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
Random rand = new Random();

System.out.print("Enter number of packets: ");
int packets = sc.nextInt();

int delivered = 0;
int lost = 0;
long totalDelay = 0;

System.out.println("\n--- TCP Transmission Simulation ---\n");

for (int i = 1; i <= packets; i++) {

int delay = rand.nextInt(50) + 50; // 50-100 ms

try {
Thread.sleep(delay);
} catch (Exception e) {}

// simulate rare packet loss (TCP retransmission)
int chance = rand.nextInt(100);

if (chance < 90) { // 90% success
System.out.println("Packet " + i + " delivered (ACK received)");
delivered++;
```

```
totalDelay += delay;
} else {
System.out.println("Packet " + i + " lost -> retransmitting...");
lost++;

// retransmission
System.out.println("Packet " + i + " retransmitted and delivered");
delivered++;
totalDelay += delay * 2;
}
}

double throughput = (double) delivered / (totalDelay / 1000.0);

System.out.println("\n--- TCP PERFORMANCE RESULT ---");
System.out.println("Total Packets: " + packets);
System.out.println("Delivered: " + delivered);
System.out.println("Lost: " + lost);
System.out.println("Total Delay: " + totalDelay + " ms");
System.out.println("Throughput: " + throughput + " packets/sec");

sc.close();
}
}
```